In [ ]:

```
PAWAR SAKSHI MAHENDRA
ROLL NO - 49
PRACTICAL NO - 12
```

In [ ]:

```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Reshape and normalize
X_train = X_train.reshape(-1, 28, 28, 1) / 255.0
X_test = X_test.reshape(-1, 28, 28, 1) / 255.0

# One-hot encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Build model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X_train, y_train, batch_size=64, epochs=10, verbose=1)

# Evaluate (FIXED ERROR HERE)
loss, accuracy = model.evaluate(X_test, y_test)

print(f"Test Loss: {loss}")
print(f"Test Accuracy: {accuracy}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datase
ts/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
```

```
C:\Users\saksh\anaconda3\Lib\site-packages\keras\src\layers\convolutional\bas
e_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument t
o a layer. When using Sequential models, prefer using an `Input(shape)` object
as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows fo
r TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. P
lease use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 36s 31ms/step - accuracy: 0.9461 - loss: 0.1788
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 30s 32ms/step - accuracy: 0.9853 - loss: 0.0481
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 30s 32ms/step - accuracy: 0.9892 - loss: 0.0345
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 34s 36ms/step - accuracy: 0.9924 - loss: 0.0256
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 31s 33ms/step - accuracy: 0.9934 - loss: 0.0215
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 32s 34ms/step - accuracy: 0.9946 - loss: 0.0176
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 31s 32ms/step - accuracy: 0.9956 - loss: 0.0133
Epoch 8/10
591/938 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - accuracy: 0.9965 - loss: 0.0104
```

In [ ]: